

Catatan Harian

Masuk untuk mulai mencatat

Nama Pengguna

Contoh: Budi

PIN (4 digit)

Masukkan PIN

Masuk

Demo: nama **budi**, PIN **1234**

127.0.0.1:5500/Soal1.html

Chat

Catatan Harian

Halo, budil Logout

Tambah Catatan

Judul catatan...

Isi catatan...

Penting

Simpan Catatan

Semua

Pribadi

Kerja

Ide

Penting

3 catatan dari total 3

Urut proposal HMIF

RAB masih salah

penting 18/5/2026 01:04

Hapus

Buat Ide

Kerjaan nambah banyak dengan nunda terus

pribadi 18/5/2026 01:03

Hapus

Tugas

tubes pwl

pribadi 18/5/2026 00:50

Hapus

127.0.0.1:5500/Soal1.html

Chat

Catatan Harian

Halo, budil Logout

Tambah Catatan

Judul catatan...

Isi catatan...

Penting

Simpan Catatan

Semua

Pribadi

Kerja

Ide

Penting

2 catatan (pribadi) dari total 3

Tugas

tubes pwl

pribadi 18/5/2026 00:50

Disematkan

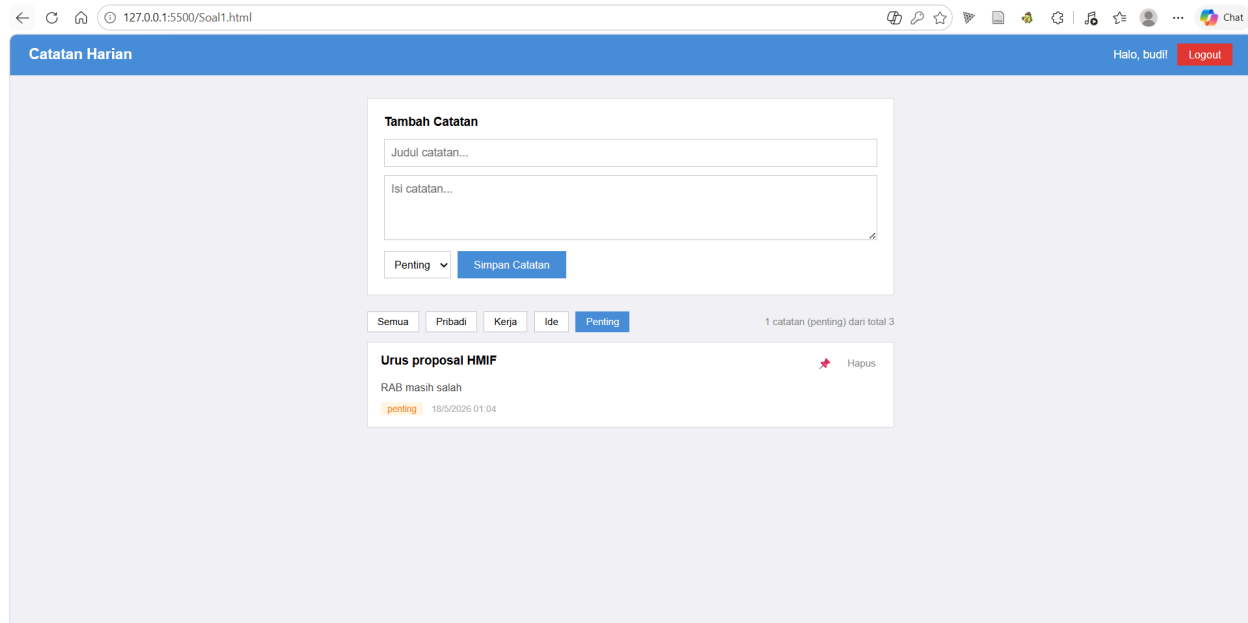
Hapus

Buat Ide

Kerjaan nambah banyak dengan nunda terus

pribadi 18/5/2026 01:03

Hapus



1. Alur Kerja Utama (Overview)

- **Sistem Login Sederhana:** Aplikasi memeriksa apakah pengguna sudah login dengan mengecek data namaUser di Local Storage. Jika belum, halaman login ditampilkan. Kredensial hardcoded: Nama **budi**, PIN **1234**.
- **Single Page Application (SPA):** Halaman tidak pernah di-refresh saat berpindah dari Login ke Halaman Utama. Transisi dilakukan dengan menyembunyikan (hide()) dan menampilkan (show()) div #loginPage dan #appPage.
- **Penyimpanan Data:** Semua catatan disimpan di memori browser (Local Storage) dalam bentuk teks JSON.

```
function getNotes() {  
  var raw = localStorage.getItem("catatanList");  
  return raw ? JSON.parse(raw) : [];  
}  
  
function saveNotes(arr) {  
  localStorage.setItem("catatanList", JSON.stringify(arr));  
}
```

Penjelasan singkat dari kode tersebut:

- **localStorage.setItem("catatanList", ...):** Ini adalah perintah utama untuk memasukkan dan menyimpan data ke dalam memori browser.
- **JSON.stringify(arr):** Kode ini bertugas mengubah data catatan kamu yang tadinya berbentuk *array* JavaScript menjadi bentuk "teks JSON" agar bisa diterima oleh memori browser.
- **localStorage.getItem("catatanList"):** Perintah ini digunakan saat aplikasi baru dibuka untuk mengambil teks JSON yang sudah tersimpan sebelumnya.

- **JSON.parse(raw)**: Kode ini berfungsi membalikkan keadaan, yaitu menerjemahkan teks JSON yang diambil dari browser kembali menjadi *array* normal supaya aplikasinya bisa menampilkan isi catatan tersebut.

2. Sistem Penyimpanan (Local Storage)

Karena tidak memakai database asli (seperti MySQL/Firebase), kita akali dengan Local Storage. Berikut fungsi utamanya:

- **getNotes()**: Mengambil data dari browser (kunci: catatanList). Karena disimpannya dalam bentuk teks JSON, kita harus mengubahnya kembali jadi array JavaScript pakai JSON.parse().
- **saveNotes(arr)**: Menerima array JavaScript, mengubahnya jadi teks dengan JSON.stringify(), lalu menyimpannya ke Local Storage.

3. Penjelasan Fitur & Logika (Event)

A. Login & Logout

```
// Validasi Login
if (nama === VALID_NAMA && pin === VALID_PIN) {
  localStorage.setItem("namaUser", nama); // Simpan sesi
  // Pindah halaman...
}
```

Saat logout, kita gunakan `localStorage.removeItem("namaUser")`. Lalu dipanggil `location.replace(location.href)` agar halaman dimuat ulang dan mencegah pengguna menekan tombol "Back" di browser untuk masuk lagi.

B. Menyimpan Catatan Baru

Saat tombol "Simpan Catatan" diklik:

1. Ambil nilai dari input judul, isi, dan kategori.
2. Lakukan validasi: jika kosong, munculkan Toast Error (`showToast`).
3. Ambil array catatan yang sudah ada lewat `getNotes()`.
4. Masukkan catatan baru ke **urutan paling atas** array menggunakan metode `unshift()` (berbeda dengan `push()` yang menaruh di bawah). Data yang disimpan termasuk ID unik dari `Date.now()`.
5. Simpan kembali ke Local Storage dan panggil `renderNotes()`.

C. Menampilkan & Memfilter Catatan (`renderNotes`)

Ini adalah fungsi yang paling sibuk. Setiap ada perubahan, fungsi ini merakit ulang HTML untuk daftar catatan:

- **Sorting (Urutan)**: Mengurutkan catatan yang disematkan (pinned) ke atas menggunakan `notes.sort(...)`.
- **Filtering**: Menggunakan `notes.filter(...)` untuk menyaring catatan sesuai kategori yang

- dipilih (Semua, Pribadi, Kerja, Ide, Penting) pada variabel `activeFilter`.
- **Render HTML:** Menggunakan `$.each()` (looping ala jQuery) untuk membaca setiap data yang lolos filter, membuat elemen `<div class="note-card">`, dan menempelkannya ke `#noteList`.

D. Event Delegation (Hapus & Pin)

Pada fitur Hapus dan Sematkan (Pin), penulisan kodenya menggunakan **Event Delegation**:

```
$(document).on('click', '.btn-hapus', function() { ... });
```

Kenapa begini? Karena elemen kartu catatan (`.note-card`) dibuat secara dinamis *setelah* halaman selesai dimuat. Jika kita menggunakan `$('.btn-hapus').on('click')` biasa, jQuery tidak akan mengenali tombol yang baru lahir tersebut. Dengan menempelkan event pada `document`, klik pada tombol dinamis tetap bisa terdeteksi.

4. Kamus Fungsi (Glosarium Cepat)

Kode / Metode	Fungsi / Kegunaan
<code>\$.trim()</code>	Membuang spasi kosong di awal dan akhir teks (agar input tidak hanya berisi spasi).
<code>e.which === 13</code>	Mendeteksi apakah tombol yang ditekan di keyboard adalah tombol Enter (berguna saat login tanpa klik tombol).
<code>\$(this).closest('.note-card')</code>	Mencari elemen pembungkus terdekat dengan class <code>.note-card</code> dari tombol yang sedang diklik. Digunakan untuk mendapatkan ID catatan yang mau dihapus/dipin.
<code>Date.now()</code>	Menghasilkan angka timestamp (milidetik sejak 1970). Di sini dimanfaatkan ganda sebagai waktu pembuatan dan ID unik (karena nilainya selalu unik setiap detiknya).